Computational Economics

# Lecture: Value Functions and the Bellman Equation

Yaolang Zhong yaolang.zhong@e.u-tokyo.ac.jp
Graduate School of Economics, The University of Tokyo

# Lecture Overview

- **The Value Function**
  - MDPs, policies, and discounted rewards
  - Expected return from a starting state
- **The Bellman Equation**
  - Recursive returns and the policy Bellman equation
  - Policy evaluation as a linear system
  - Optimal value and Bellman optimality
- **Value Function Iteration**
  - The Bellman operator and convergence
  - Iteration, stopping criteria, and policy recovery
  - Finite-horizon backward induction

# Recap: A Markov Decision Process

A time-homogeneous discounted **MDP** is specified by $(\mathcal{S}, \{\mathcal{A}(s)\}_{s\in\mathcal{S}}, P, r, \beta)$.

- **State space $\mathcal{S}$:** free cells $\mathcal{S} = \{\mathrm{S}, \mathrm{G}, 1, \ldots, 45\}$, including G. Walls are excluded.
- **Actions $\mathcal{A}(s)$:** up, down, left, right.
- **Transition kernel $P(s' \mid s, a)$:** probability 1 for the cell reached by action $a$, and 0 for all other cells. For example, $P(21 \mid \mathrm{S}, \text{down}) = 1$. A wall or boundary keeps the current cell; G is absorbing.
- **Mean reward $r(s, a)$:** $-1$ per move from outside G (including entry into G); zero at G.
- **Discount factor $\beta \in (0, 1)$:** weights future rewards.

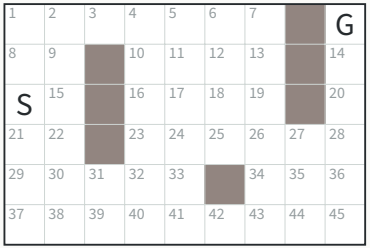



Figure 1: Maze navigation.

# Recap: A Policy Chooses an Action at Each State

A **stationary deterministic policy** is a rule $\sigma$ with $\sigma(s) \in \mathcal{A}(s)$: at cell $s$, take action $a = \sigma(s)$.

$\sigma_A = (\text{S} \to \text{down},\ \text{cell } 21 \to \text{down},\ \text{cell } 29 \to \text{right}, \ldots)$

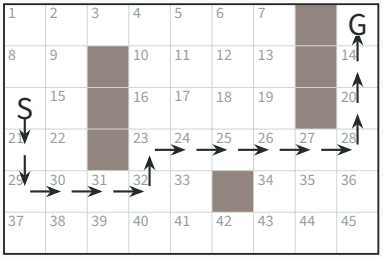



Figure 2: Policy $\sigma_A$.

$\sigma_B = (\text{S} \to \text{up},\ \text{cell } 8 \to \text{up},\ \text{cell } 1 \to \text{right}, \ldots)$

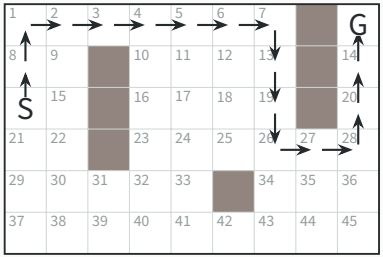



Figure 3: Policy $\sigma_B$.

# From a State and a Policy to a Reward Stream

Start at $s_t = s$. At future date $t + \tau - 1$, choose $a_{t+\tau-1} = \sigma(s_{t+\tau-1})$, for $\tau = 1, 2, \ldots$.

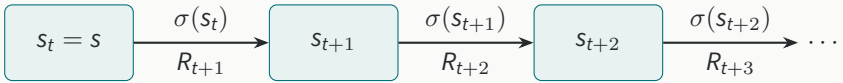



Figure 4: States, actions, and rewards.

The **discounted accumulated reward**, or **return**, from date $t$ is

$$G_t = R_{t+1} + \beta R_{t+2} + \cdots + \beta^{\tau-1} R_{t+\tau} + \cdots = \sum_{\tau=1}^{\infty} \beta^{\tau-1} R_{t+\tau}.$$

For the two maze routes, take $\beta = 0.9$:

$$G_t^{\sigma_A} = -\sum_{\tau=1}^{14} 0.9^{\tau-1} \approx -7.71, \qquad G_t^{\sigma_B} = -\sum_{\tau=1}^{16} 0.9^{\tau-1} \approx -8.15.$$

## Discounting and a Finite Return

Assume $|R_{t+j}| \le \bar{r} < \infty$ for every $j \ge 1$, with $0 < \beta < 1$.

Over the next $\tau$ periods,

$$\left| G_t^{(\tau)} \right| = \left| \sum_{j=1}^{\tau} \beta^{j-1} R_{t+j} \right| \le \sum_{j=1}^{\tau} \beta^{j-1} |R_{t+j}|$$
$$\le \bar{r} \sum_{j=1}^{\tau} \beta^{j-1} = \frac{1-\beta^{\tau}}{1-\beta}\, \bar{r}.$$

As $\tau \to \infty$, $\beta^{\tau} \to 0$, so the infinite-horizon return satisfies

$$|G_t| \le \lim_{\tau \to \infty} \frac{1-\beta^{\tau}}{1-\beta}\, \bar{r} = \frac{\bar{r}}{1-\beta}.$$

# Example: Two Cake-Eating Policies

## Example: Cake Eating

Start with two units of cake. For this example, restrict stock and consumption to whole units:

$$k \in \{0, 1, 2\}, \quad c \in \{0, \ldots, k\}, \quad k' = k - c, \quad r(k, c) = \sqrt{c}, \quad \beta = 0.9.$$

**Consume everything:** $\sigma_A(k) = k$

$$k_t, k_{t+1}, k_{t+2}: \quad 2 \longrightarrow 0 \longrightarrow 0$$

$$R_{t+1}, R_{t+2}, \ldots: \quad \sqrt{2},\ 0,\ 0, \ldots$$

$$G_t = \sqrt{2} \approx 1.414.$$

**One unit at a time:** $\sigma_B(k) = \min\{1, k\}$

$$k_t, k_{t+1}, k_{t+2}: \quad 2 \longrightarrow 1 \longrightarrow 0$$

$$R_{t+1}, R_{t+2}, \ldots: \quad 1,\ 1,\ 0, \ldots$$

$$G_t = 1 + 0.9 = 1.9.$$

A smaller reward today can produce a larger accumulated reward. These paths are deterministic, so their returns can be calculated directly.

## The Value Function of a Policy

For time-homogeneous dynamics and a stationary policy, the **value function under policy** $\sigma$ assigns each state its expected return, independently of the starting date $t$:

$$V^{\sigma}(s) = \mathbb{E}_{\sigma}[G_t \mid s_t = s] = \mathbb{E}_{\sigma}\left[\sum_{\tau=1}^{\infty} \beta^{\tau-1} R_{t+\tau} \,\middle|\, s_t = s\right].$$

- $s$ fixes the initial state; $\sigma$ specifies actions at every future state. The expectation averages over the resulting paths and rewards.
- Since $r(s, a)$ is the conditional mean reward, boundedness also gives

$$V^{\sigma}(s) = \mathbb{E}_{\sigma}\left[\sum_{\tau=1}^{\infty} \beta^{\tau-1} r(s_{t+\tau-1}, \sigma(s_{t+\tau-1})) \,\middle|\, s_t = s\right].$$

- For the cake example, $V^{\sigma_A}(2) = \sqrt{2}$ and $V^{\sigma_B}(2) = 1.9$.

## The Return Has a Recursive Form

From any date $t$, define the return using that date as the discounting origin:

$$G_t = R_{t+1} + \beta R_{t+2} + \beta^2 R_{t+3} + \cdots .$$

Separate the first reward from all later rewards:

$$\begin{aligned} G_t &= R_{t+1} + \beta\big(R_{t+2} + \beta R_{t+3} + \cdots\big) \\ &= R_{t+1} + \beta G_{t+1}. \end{aligned}$$

- The future return is the same object, starting one period later.
- With time-homogeneous dynamics and a stationary policy,

$$\mathbb{E}_\sigma[G_{t+1} \mid s_{t+1} = s'] = V^\sigma(s').$$

This lets us replace an infinite reward sequence with a one-step equation.

## The Bellman Equation for a Fixed Policy

Take conditional expectations of $G_t = R_{t+1} + \beta G_{t+1}$:

$$\begin{aligned} V^\sigma(s) &= \mathbb{E}_\sigma[R_{t+1} + \beta G_{t+1} \mid s_t = s] \\ &= r(s, \sigma(s)) + \beta \mathbb{E}[V^\sigma(s_{t+1}) \mid s_t = s,\ a_t = \sigma(s)]. \end{aligned}$$

The second equality uses the law of iterated expectations and the Markov structure.

For a finite state space, the **policy Bellman equation** is

$$V^\sigma(s) = r(s, \sigma(s)) + \beta \sum_{s' \in \mathcal{S}} P(s' \mid s, \sigma(s)) V^\sigma(s').$$

Current expected reward plus discounted expected continuation value. For a continuous state space, replace the sum by an integral with respect to $P(ds' \mid s, \sigma(s))$.

## Policy Evaluation as a Linear System

From here, assume $\mathcal{S}$ and each nonempty $\mathcal{A}(s)$ are finite.

Fix $\sigma$. Recall its induced transition matrix and reward vector:

$$P^{\sigma}_{ij} = P(j \mid i, \sigma(i)), \qquad r^{\sigma}_{i} = r(i, \sigma(i)).$$

Treat $V^{\sigma}$ and $r^{\sigma}$ as **column vectors**. Then

$$V^{\sigma} = r^{\sigma} + \beta P^{\sigma} V^{\sigma} \quad \Longleftrightarrow \quad (I - \beta P^{\sigma}) V^{\sigma} = r^{\sigma}.$$

- $P^{\sigma}$ is row-stochastic, so $\beta < 1$ makes $I - \beta P^{\sigma}$ invertible:

$$V^{\sigma} = \sum_{\tau=1}^{\infty} \beta^{\tau-1} (P^{\sigma})^{\tau-1} r^{\sigma} = (I - \beta P^{\sigma})^{-1} r^{\sigma}.$$

- In code, solve the linear system rather than forming the inverse.

# Example: Income Along an Employment Chain

**Example: Employment and Unemployment**

Take the earlier employment dynamics as given, and attach income 0 in U and 1 in E. There is no action choice in this calculation.

$$P = \begin{pmatrix} 0.8 & 0.2 \\ 0.05 & 0.95 \end{pmatrix}, \qquad r = \begin{pmatrix} 0 \\ 1 \end{pmatrix}, \qquad \beta = 0.9.$$

The value of the income stream solves

$$V(\mathrm{U}) = 0 + 0.9\big[0.8V(\mathrm{U}) + 0.2V(\mathrm{E})\big],$$
$$V(\mathrm{E}) = 1 + 0.9\big[0.05V(\mathrm{U}) + 0.95V(\mathrm{E})\big].$$

$$V(\mathrm{U}) = \frac{72}{13} \approx 5.538, \qquad V(\mathrm{E}) = \frac{112}{13} \approx 8.615.$$

- Unemployment has positive value because future employment is possible.
- Employment is worth less than $1/(1-0.9) = 10$, because job loss is possible.

## Iterative Policy Evaluation

We can also compute $V^\sigma$ by repeated substitution, keeping $\sigma$ fixed:

$$v^{(n+1)} = r^\sigma + \beta P^\sigma v^{(n)}, \qquad v^{(0)} = 0.$$

- Use the old vector $v^{(n)}$ for every state in a sweep.
- After $n$ sweeps, $v^{(n)}$ is the expected return from the first $n$ rewards under $\sigma$, with zero value beyond them.

For the employment example:

$$v^{(0)} = \begin{pmatrix} 0 \\ 0 \end{pmatrix}, \quad v^{(1)} = \begin{pmatrix} 0 \\ 1 \end{pmatrix}, \quad v^{(2)} = \begin{pmatrix} 0.180 \\ 1.855 \end{pmatrix}, \quad v^{(n)} \longrightarrow \begin{pmatrix} 5.538\ldots \\ 8.615\ldots \end{pmatrix}.$$

$n$ counts computational iterations. The policy being evaluated does not change.

## From Policy Values to the Optimal Value

Let $\Sigma$ be the set of feasible stationary deterministic policies. Define the **optimal value function** by

$$V^*(s) = \max_{\sigma \in \Sigma} V^\sigma(s).$$

- A policy $\sigma^*$ is **optimal** if it achieves this value at every state:

$$V^{\sigma^*}(s) = V^*(s) \qquad \text{for every } s \in \mathcal{S}.$$

- In a finite, time-homogeneous discounted MDP, such a stationary deterministic policy exists. Allowing randomized or history-dependent policies cannot improve the value.
- The policy may be nonunique when several actions are equally good.

The task now is to choose a policy, as well as evaluate the rewards it generates.

# The Bellman Optimality Equation

The **principle of optimality**: after the next state is observed, an optimal policy uses an optimal continuation from that state.

Choose the current action $a$, then continue optimally. Its expected return is

$$r(s,a) + \beta \sum_{s'} P(s' \mid s,a) V^*(s').$$

Maximizing over the feasible current actions gives the **Bellman optimality equation**:

$$V^*(s) = \max_{a \in \mathcal{A}(s)} \left\{ r(s,a) + \beta \sum_{s'} P(s' \mid s,a) V^*(s') \right\}.$$

- The current action is chosen before the random next state is observed.
- Future actions can respond to that realized state. $V^*(s')$ includes all those later choices.

# Example: The Cake-Eating Bellman Equation

**Example: Cake Eating**

Use the same whole-unit model, $k \in \{0, 1, 2\}$ and $\beta = 0.9$:

$$V^*(k) = \max_{c \in \{0,\dots,k\}} \{\sqrt{c} + 0.9V^*(k-c)\}.$$

At zero stock, $V^*(0) = 0.9V^*(0)$, so $V^*(0) = 0$. At one unit,

$$V^*(1) = \max\{0.9V^*(1),\ 1\} = 1.$$

At two units, check all three feasible actions:

$$V^*(2) = \max\{\underbrace{0.9V^*(2)}_{c=0},\ \underbrace{1 + 0.9V^*(1)}_{c=1},\ \underbrace{\sqrt{2}}_{c=2}\} = 1.9.$$

- At this solution, the three candidate values are $1.71, 1.9, 1.414\ldots$.
- Thus $\sigma^*(2) = 1$, $\sigma^*(1) = 1$, and $\sigma^*(0) = 0$.

## The Bellman Operator

An **operator** maps a candidate value function $v$ to a new function.

$$(Tv)(s) = \max_{a \in \mathcal{A}(s)} \left\{ r(s,a) + \beta \sum_{s'} P(s' \mid s,a) v(s') \right\}.$$

- $v(s')$ is our current assessment of future states. $Tv$ updates that assessment using the best current action.
- Holding a policy fixed instead gives

$$(T^\sigma v)(s) = r(s,\sigma(s)) + \beta \sum_{s'} P(s' \mid s,\sigma(s)) v(s').$$

- A **fixed point** is unchanged by its operator:

$$V^* = TV^*, \qquad V^\sigma = T^\sigma V^\sigma.$$

## Why the Iteration Converges

Use the **maximum norm** $\|v - w\|_\infty = \max_s |v(s) - w(s)|$. For any state and action,

$$\left| \beta \sum_{s'} P(s' \mid s, a) \big[ v(s') - w(s') \big] \right| \le \beta \|v - w\|_\infty .$$

Taking the maximum over actions preserves this bound, since $|\max_a x_a - \max_a y_a| \le \max_a |x_a - y_a|$. Hence

$$\|Tv - Tw\|_\infty \le \beta \|v - w\|_\infty .$$

- $T$ is a **contraction**: the new distance is at most $\beta < 1$ times the old distance.
- On the finite-dimensional value space, it has a unique fixed point $V^*$, and

$$\|T^n v^{(0)} - V^*\|_\infty \le \beta^n \|v^{(0)} - V^*\|_\infty .$$

- The same argument applies to $T^\sigma$ and its fixed point $V^\sigma$.

## Value Function Iteration

Given the finite MDP and a desired value-error tolerance $\varepsilon > 0$:

1. Choose $v^{(0)}$, for example zero, and set $n = 0$.
2. Update every state using the previous vector:

$$v^{(n+1)}(s) = (Tv^{(n)})(s) = \max_{a \in \mathcal{A}(s)} \left\{ r(s,a) + \beta \sum_{s'} P(s' \mid s,a) v^{(n)}(s') \right\}.$$

3. Calculate the **Bellman residual** of $v^{(n)}$:

$$d_n = \|Tv^{(n)} - v^{(n)}\|_\infty = \|v^{(n+1)} - v^{(n)}\|_\infty.$$

4. If $d_n \leq (1-\beta)\varepsilon$, return $v^{(n)}$. Otherwise increase $n$ and repeat.

The contraction inequality implies

$$\|v^{(n)} - V^*\|_\infty \leq \frac{d_n}{1-\beta} \leq \varepsilon.$$

This bound assumes exact expectations and maximization in the specified finite model.

# Example: Value Iteration for Cake Eating

**Example: Cake Eating**

Start from $v^{(0)}(k) = 0$, using $\beta = 0.9$.

| | Stock 0 | Stock 1 | Stock 2 |
|---|---|---|---|
| $v^{(0)}$ | 0 | 0 | 0 |
| $v^{(1)} = Tv^{(0)}$ | 0 | 1 | $\sqrt{2} \approx 1.414$ |
| $v^{(2)} = Tv^{(1)}$ | 0 | 1 | 1.9 |
| $v^{(3)} = Tv^{(2)}$ | 0 | 1 | 1.9 |

- The first update assigns zero continuation value, so consuming everything looks best.
- In the second update, saving one unit has value:

$$1 + 0.9v^{(1)}(1) = 1.9 > \sqrt{2}.$$

- A further update leaves the vector unchanged: the Bellman residual is zero.

# A Finite Horizon: Backward Induction

If decisions end at date $H-1$, let $h(s_H)$ be a terminal payoff at date $H$. The objective from $s_t = s$ is

$$\max_{\sigma_t,\ldots,\sigma_{H-1}} \mathbb{E}\left[\sum_{\tau=1}^{H-t} \beta^{\tau-1} R_{t+\tau} + \beta^{H-t} h(s_H) \,\middle|\, s_t = s\right].$$

**Backward induction** starts at the terminal date and works toward the present:

$$V_H(s) = h(s),$$

$$V_t(s) = \max_{a \in \mathcal{A}(s)} \left\{ r(s,a) + \beta \sum_{s'} P(s' \mid s,a) V_{t+1}(s') \right\}, \quad t = H-1, \ldots, 0.$$

- The remaining horizon changes with $t$; optimal decision rules can depend on time.
- With $h = 0$, $T^H 0$ is the value with $H$ decisions remaining. This also explains the earlier cake iterations.

## Recovering a Policy from Values

Once we have $V^*$, choose an action that attains its Bellman maximum:

$$\sigma^*(s) \in \arg\max_{a \in \mathcal{A}(s)} \left\{ r(s,a) + \beta \sum_{s'} P(s' \mid s,a) V^*(s') \right\}.$$

- A **greedy policy with respect to** $v$ uses the same expression with a candidate value $v$ in place of $V^*$.
- With an approximate value, the resulting policy is a candidate. Evaluate it by solving

$$(I - \beta P^\sigma) V^\sigma = r^\sigma.$$

- The value function summarizes future rewards; the policy specifies the action to take. Computing both makes the solution usable.

# References and Further Reading


1. **Richard S. Sutton and Andrew G. Barto (2018), Reinforcement Learning: An Introduction. 2nd ed. MIT Press. [Book]**
   - Chapter 3: Finite Markov Decision Processes (Sections 3.3, 3.5 and 3.6)
   - Chapter 4: Dynamic Programming (Sections 4.1 and 4.4)
2. **Thomas J. Sargent and John Stachurski, QuantEcon. [Online lectures]**
   - Discrete State Dynamic Programming
3. **Feng (2026), Foundations of Macroeconomics: ECON 282E, Fall 2026. [Online course]**
   - Session 1, Deck 1.B2: Dynamic Programming: Why It Works and How It Computes